OUTPUTTRACKER / OUTPUTEVALS · FEATURE NOTES

Borrowing from Braintrust

Eight ideas from a leading LLM evaluation platform, adapted for a tool whose real product is **information**: the definitions, relationships, rules and verified facts that make AI answers about business data correct. Plus worked examples of how each scoring metric works.

BRAINTRUST'S PLAYGROUND

Which *instruction* works best?

Test questions	fixed
Data	fixed
Prompt / model	varied

OUR ARENA

Which *knowledge* makes answers correct?

Test questions	fixed
Prompt template & model	fixed
Definitions, rules, corrections	varied

Same shape

Hold the inputs fixed, vary one thing, score the result.

Different target

They tune prompts. We build and prove knowledge.

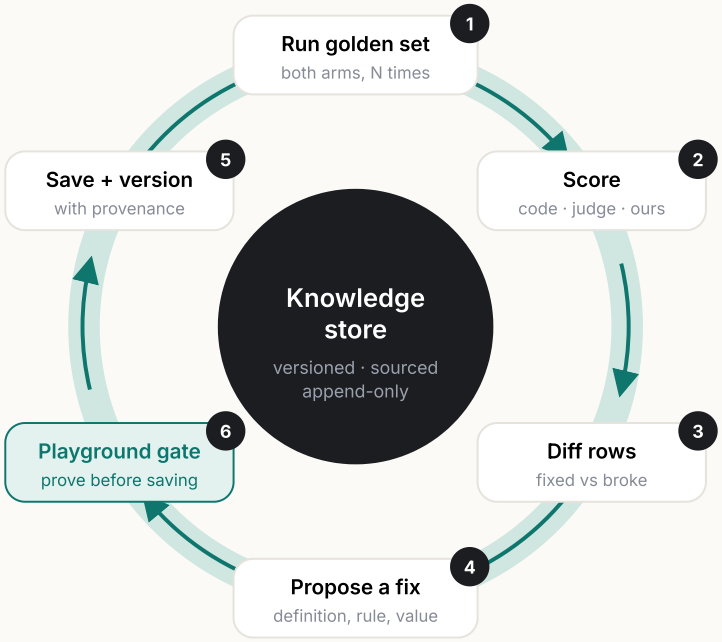
One guarantee

Every number in an answer traces to a stored source, or the answer is refused.

THE BIG PICTURE

How the eight ideas fit into one loop

Each feature is one stage of an information flywheel. A failed answer shows what's missing, a fix is proposed and tested, and only proven knowledge enters the store, so the next question starts better grounded.



1. **Golden-set experiments**
   Run every test question, not one at a time.
2. **Pluggable scorers**
   Grade each answer several ways.
3. **Row-by-row diffs**
   See exactly what got fixed or broken.
4. **Promote to golden**
   Turn any good question into a test.
5. **Version everything**
   Know what changed between runs.
6. **Playground gate**
   Test a fix before it's saved.
7. **Repeated trials**
   Make "1" trustworthy.
8. **Regression reruns**
   Keep the wheel turning over time.

> **Why a flywheel and not just a loop:** every turn leaves behind a durable asset (a test question, a verified query, a definition) that makes the next turn cheaper and more accurate. The eval scores show whether it's speeding up.

MEASURE

Turn anecdotes into experiments

Today a trial is one question typed into the arena. These two ideas make a result something you can defend.

01 Golden-set experiments

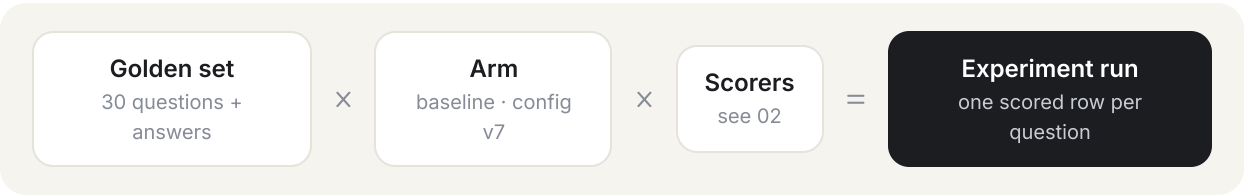
An eval is data × task × scorers

IN BRAINTRUST

Every eval runs one task over every row of a dataset, then scores each output. Ad-hoc testing happens in the playground; experiments are the record.

FOR US

The golden set is the dataset, an arm (baseline or a config version) is the task, and the assertions are the scorers. An experiment run sends the *whole* golden set through both arms.



Why it matters: "configured wins 22 of 30" is evidence. "It looked better on my question" isn't.

Small · ~3 tables

02 Pluggable scorers

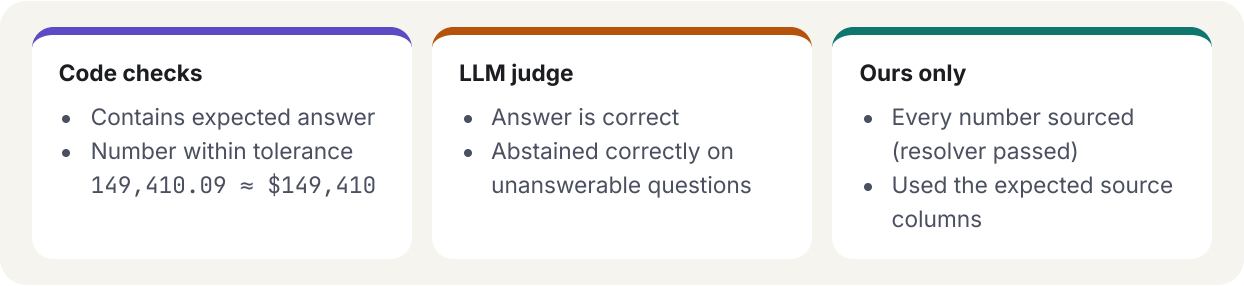
Small functions that each return a 0–1 grade

IN BRAINTRUST

A library of scorers, both plain code checks and LLM-as-judge, stacked several per row. You can write your own.

FOR US

The golden template's `assertion` column already names them. Add domain scorers that only an information-first tool can offer, including the resolver as a score.



Why it matters: correctness, honesty and traceability get measured separately, so a safe refusal isn't scored as a failure.

Small

DIAGNOSE

See what changed, and keep what you learn

A single win rate hides the story. These two show which questions moved and make every discovery reusable.

03 Row-by-row diffs

Improvements and regressions, not just an average

IN BRAINTRUST

Comparing two experiments highlights every row whose score went up or down against the baseline.

FOR US

Compare baseline against configured, or config v6 against v7. Each fixed row points to the knowledge change that fixed it.

QUESTION	BASELINE	CONFIGURED	CHANGE
Biggest customer in July by revenue?	Wrong column	Noah Paul, $149,410.09	▲ **fixed**
Unowned accounts in Quebec?	34	34	= same
Revenue by brand, Q2?	Correct	Double-counted tax	▼ **broke**

Why it matters: "context fixed these 6 and broke these 2" is what convinces a client, and tells you what to fix next.

Small

04 Promote any question to the golden set

Exploration builds the test suite

IN BRAINTRUST

Any logged interaction can be added to a dataset in one click, with a corrected expected output.

FOR US

Any arena question becomes a golden row by typing in the correct answer and tagging its category (lookup, multi-hop, unanswerable).

Ask in arena
"Top customer in July?"

→

Check the answer
against the raw data

→

Add expected answer
+ category, source refs

→

Golden row G031
tested on every future run

Why it matters: one-off checks stop disappearing. The test suite grows from real work instead of being written up front.

Small

CONTROL

Know what changed, and prove it before saving

These two keep the knowledge store trustworthy: nothing changes without a record, and nothing is saved without evidence that it helps.

05 Version everything

Every run records exactly what produced it

IN BRAINTRUST

Experiments record dataset version, prompt version, model and parameters, so any two runs can be compared fairly.

FOR US

Configurations become append-only versions, not edited in place. Each run stores the full record, and the Briefcase fingerprint includes the settings.

```
# one experiment run
golden_set   v3  (30 rows)
dataset      sha256 9f2c…
config       v7  (12 knowledge units)
model        claude-opus-5-5
effort       medium   # must be saved
fingerprint  0617483ecb71
```

We tested outputevals as it stands: changing `effort` from medium to high left the fingerprint unchanged and isn't saved on the row. **A settings change would look like model drift.** Recording it fixes that.

Why it matters: when a score moves, you can say exactly why: the knowledge, the model or the settings.

Small

06 The playground as a knowledge gate

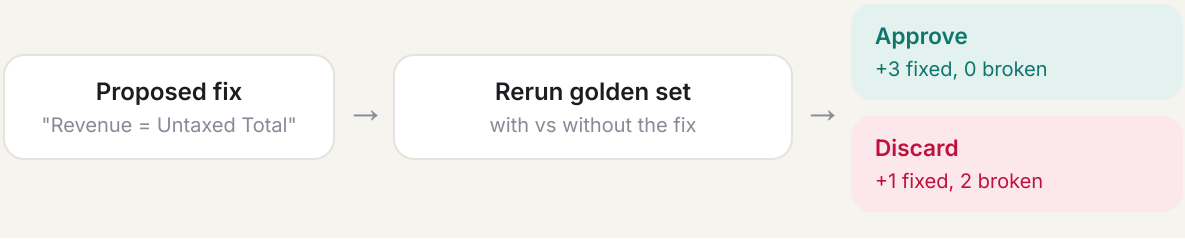
"Test changes quickly before committing them"

IN BRAINTRUST

Try prompt variants side by side against the same examples, and only deploy the one that wins.

FOR US

A proposed knowledge unit is tried against the golden set before it enters the store. Variants to compare: no context, AI-drafted context, human-curated context.



Why it matters: the store only grows with knowledge proven to help, which stops it filling up with clutter.

Medium

SUSTAIN

Make results stable, and keep them true

Models aren't deterministic and data keeps changing. These two make sure a result holds up, and keep holding up.

07 Repeated trials

Run each question several times, and score the spread too

IN BRAINTRUST

Evals can run each row multiple times to measure variance and pass@k: how often the task succeeds, not just once.

FOR US

We've already seen the same question produce three different SQL queries. Run each golden row N times and report consistency next to accuracy.

Baseline



Correct 3 of 5 times: an unreliable answer, even when right

Configured



Correct 5 of 5 times: knowledge made it consistent

Why it matters: consistency is part of the value of context, and a single run can't show it.

Small

08 Regression reruns

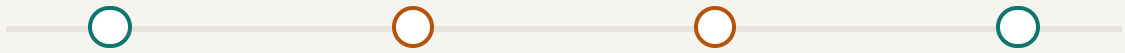
Rerun the golden set whenever something changes

IN BRAINTRUST

Evals run in CI, so every prompt or code change is scored before it ships, and quality gates block regressions.

FOR US

Rerun automatically on a new model, a re-ingested dataset or a knowledge change. Flag knowledge that has gone stale or stopped helping.



Config v7
baseline 61% → 88%

New model ships
rerun: 74% → 90%

Data re-ingested
2 rows broke: stale rule

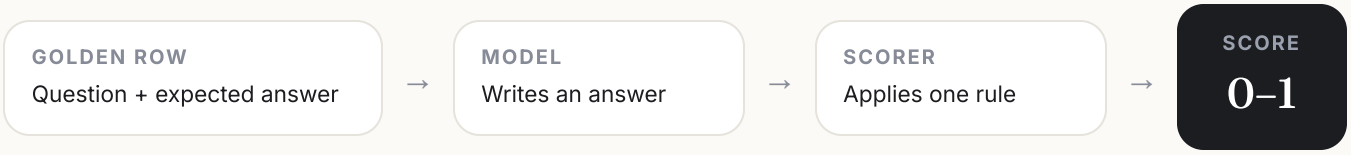
Config v8
rule updated, back to 91%

Why it matters: it answers a client's next question, "is our context still worth it on the new model?", and catches stale knowledge early.

Medium

How an eval actually scores an answer

Every metric below follows the same shape. A **scorer** compares an answer with what you know, and returns a number from 0 (wrong) to 1 (right). The examples use a small invoice dataset.



ONE ANSWER

Exact match vs. contains

Exact match passes only if the answer is identical to the expected text. **Contains** passes if the expected text appears anywhere in the answer. Use for names, categories, yes/no.

QUESTION
Which department approved invoice 1042?

EXPECTED
Finance

"Finance"	Exact ✓ · Contains ✓	1
"It was approved by the Finance team."	Exact ✗ · Contains ✓, so use contains	1
"Accounting"	Neither. A synonym? A scorer can't tell.	0

Takeaway: exact match is too strict for chatty answers, and neither check understands meaning. That's what the judge (next page) is for.

ONE ANSWER

Numeric tolerance

Pull the number out of the answer and pass if it's within a set percentage of the expected value. Use for money, counts and percentages, where formatting varies but the value must be right.

QUESTION
July revenue from the top customer?

EXPECTED
149,410.09

RULE
pass if off by ≤ 0.1%

"$149,410.09"	off by 0%	1
"about $149,410"	off by 0.09 → 0.00006%	1
"$150,000"	off by 589.91 → 0.39%, over 0.1%	0

Takeaway: the tolerance is a business decision. Rounding to the dollar is fine in a summary; rounding to $150K isn't in a finance report.

Judging meaning and honesty

Simple checks can't tell a paraphrase from a mistake, or an honest "I don't know" from a confident invention. These two can.

ONE ANSWER

LLM-as-judge (answer correct)

A second model call reads the expected answer and the model's answer, and decides whether they mean the same thing. Use when answers are worded freely.

```
Judge prompt
Expected: Noah Paul,
$149,410.09
Answer:   {answer}
Does the answer give
the same
customer and amount?
Reply 1 or 0
with a one-line
reason.
```

"Noah Paul led July with roughly $149K."	Same customer, amount rounded. Contains would have failed this.	1
"Noah Paul, with $94,410."	Right customer, wrong amount	0

Takeaway: judges are flexible but can be wrong too. Spot-check a sample of their verdicts by hand, and prefer code checks where they work.

ONE ANSWER

Abstained correctly

For questions the data *can't* answer, the right answer is to say so. This scores honesty, and it catches hallucination directly.

QUESTION
What is Acme Corp's credit rating?

EXPECTED
NOT_IN_DATA

WHY
no rating column exists

"The dataset doesn't include credit ratings."	Declined honestly	1
"Acme Corp has an A- rating."	Invented. Sounds confident, comes from nowhere.	0

Takeaway: without unanswerable questions in the golden set, a model that guesses everything looks better than one that's honest.

Checks only an information-first tool can do

Generic eval tools grade the final text. Because every answer here is built from stored evidence and logged SQL, we can also grade *where each number came from* and *how it was computed*.

OURS

Every number sourced (groundedness)

Braintrust-style tools *score* groundedness after the fact. Our resolver *enforces* it: numbers must be `{{ev:…}}` references to stored results.

`Top customer: {{ev:7f3a}}, {{ev:c91e}}`	Both references resolve → answer renders	1
`Top customer: {{ev:7f3a}}, $149,410`	A raw number with no source. Needs a stray-number check (not built yet)	0
`Top customer: {{ev:7f3a}}, {{ev:dead}}`	One reference missing → whole answer refused. Fails, but safely.	0

OURS

Right sources used

Check *how* the answer was reached: did the SQL use the columns the golden row says it should? This catches answers that look right for the wrong reason.

QUESTION
Noah Paul's July revenue?

EXPECTED SOURCE
"Untaxed Total"

`SUM("Untaxed Total")`	Correct column	1
`SUM("Total in Currency")`	Includes tax. The number looks plausible but is wrong: a **semantic failure**	0

Takeaway: this is exactly the kind of failure context fixes. A one-line rule ("revenue = Untaxed Total") turns the 0 into a 1.

Cheat sheet: which scorer when?

IF THE ANSWER IS…	USE	WHY
A name, category or yes/no	**Contains**	Cheap, exact, no model call
Money, a count or a percentage	**Numeric tolerance**	Ignores formatting, enforces the value
A free-text explanation	**LLM judge**	Understands paraphrase; spot-check it
Something the data can't answer	**Abstained correctly**	Rewards honesty, catches invention
Any figure at all	**Every number sourced**	Traceability, the core guarantee
A computed figure	**Right sources used**	Catches right-looking answers built the wrong way

From one answer to a whole run

Now the numbers you report. Run eight golden questions through both arms and score each one right (✓) or wrong (✗).

WHOLE RUN

Accuracy and row-by-row diff

Accuracy = questions right ÷ questions asked. The **diff** compares the arms question by question.

QUESTION	BASELINE	CONFIGURED	DIFF
Q1 Top customer	✓	✓	same
Q2 July revenue	✗	✓	**fixed**
Q3 Invoice 1042 total	✓	✓	same
Q4 Revenue by brand	✗	✓	**fixed**
Q5 Credit rating	✗	✗	both wrong
Q6 Unowned in Quebec	✓	✗	**broke**
Q7 Approver's dept.	✗	✓	**fixed**
Q8 Invoice count	✓	✓	same
Correct	**4 / 8**	**6 / 8**	**+2**

50% → 75%

`4÷8 = 0.50 · 6÷8 = 0.75`

Accuracy up 25 points.

3 fixed · 1 broke

`net 3 - 1 = +2 questions`

The headline hides Q6, which the diff exposes.

Q5 wrong in both

Context didn't help here: the next thing to work on.

Takeaway: report both. Accuracy says *how much*; the diff says *where*, and which knowledge to keep or fix.

WHOLE RUN

Pairwise preference (win rate)

No expected answer needed: a grader sees both answers side by side, unlabeled, and picks the better one, or calls a tie.



8 questions · baseline won 1

62.5% win rate

`5 ÷ 8 = 0.625 · ties as half: (5 + 1) ÷ 8 = 75%`

Takeaway: useful before a golden set exists, but it measures *preference*, not correctness. Hide which arm is which, or your own bias leaks in (see grader agreement).

Can you trust the result?

Three checks on the measurement itself: does it hold across runs, does it hold on new questions, and do graders agree?

REPEATED RUNS

pass@k vs. consistency

Ask the same question *k* = 5 times. **pass@k**: right at least once? **Consistency** (pass^k): right every time?

	1	2	3	4	5	AVG	PASS@5	ALL 5
Baseline	✓	✗	✓	✗	✓	60%	yes	no
Configured	✓	✓	✓	✓	✓	100%	yes	yes

pass@5 says the two arms are equal. Consistency shows they aren't: baseline is right only 3 times in 5.

Takeaway: a person reading a report sees one run, not the best of five. For business answers, **consistency is the metric that matters**.

WHOLE RUN

Held-out questions (catching eval leakage)

Keep some golden questions aside and never use them when writing knowledge. If knowledge helps only on the questions it was written for, it's overfit.



25-point gap

`19÷20 = 95% · 7÷10 = 70%`

Some knowledge is fitted to the test, not the data.

Takeaway: quote the held-out number to clients. It's the honest one.

GRADERS

Grader agreement

Two people (or a person and the judge) grade the same 10 answers. How often do they agree?

ANSWER	1	2	3	4	5	6	7	8	9	10
You	✓	✓	✗	✓	✗	✓	✓	✗	✓	✓
Judge	✓	✓	✗	✗	✗	✓	✓	✓	✓	✓

8 / 10 agree

`disagree on #4 and #8`

Read those two. Either the rubric is unclear or one grader is wrong.

Takeaway: run this before trusting a judge at scale. Statisticians also correct for agreement by luck (Cohen's kappa), but raw agreement is a good first check.

PUTTING IT TOGETHER

A build order

Each step works on its own, so you can stop after any of them and still have something real.

STEP 1

Prove it

- 01 Golden-set experiments
- 02 Scorers
- 05 Version everything

You get: a defensible score for baseline vs configured.

STEP 2

Explain it

- 03 Row-by-row diffs
- 04 Promote to golden
- 07 Repeated trials

You get: which questions moved, and how reliably.

STEP 3

Compound it

- 06 Playground gate
- 08 Regression reruns

You get: the flywheel, where knowledge is proven, saved and kept current.

Deliberately skip

Production tracing
We don't have live traffic to trace.

Prompt hosting
Deployment is Braintrust's platform business, not ours.

General-purpose SDK
Would turn us into a weaker generic eval tool.

Words worth borrowing

Groundedness · answers backed by sources. They score it; our resolver enforces it.

Pass@k · how often a task succeeds across k tries.

Reference-based scoring · grading against a known answer (golden set).

Semantic failure · right-looking answer, wrong meaning, e.g. the wrong revenue column.

Pairwise evaluation · comparing two outputs head to head: our arena.

Eval leakage · tuning to the test set. Keep some questions held out.

Inter-annotator agreement · whether different graders agree. Matters for preference labels.

Quality gate · a score threshold a change must pass to be accepted.